University of Padova

Department of Mathematics ”Tullio Levi-Civita”

Bachelor's Degree in Computer Science

Explainable Machine Learning through Large Language Models: Analysis and Prompt Design

Bachelor's Thesis

Supervisor

Prof. Michele Scquizzato

Undergraduate

Lorenzo Soligo

ID number 2101057

ACADEMIC YEAR 2025-2026

Lorenzo Soligo: *Explainable Machine Learning through Large Language Models: Analysis and Prompt Design*, Bachelor's Thesis, © June 2026

TO DO

—— TO DO

Dedicato a ...

Abstract

As Machine Learning (ML) models become increasingly prevalent in business applications, ensuring their transparency and reliability through Explainable Artificial Intelligence (XAI) is critical. This project, conducted in collaboration with Zucchetti S.p.A., investigates the interpretability and comprehensibility of various machine learning algorithms.

The research systematically evaluates a spectrum of predictive models, ranging from fundamental Regression and Classification models—such as Linear Regression, Logistic Regression, Support Vector Machines, and Decision Trees—to complex ensemble methods, including Random Forest, XGBoost and Symbolic Regression. The analysis explores both intrinsic model transparency and post-hoc explainability techniques, utilizing metrics like SHapley Additive exPlanations (SHAP) to interpret predictions and feature importance. Furthermore, the methodology encompasses the examination of mathematical foundations, internal knowledge representations, and the impact of ensemble techniques on both model performance and explainability. Real-world validation is performed using datasets such as Student Salary Prediction.

A primary objective of this work is bridging the gap between technical accuracy and human-understandable explanations for non-expert stakeholders. To achieve this, the project leverages advanced Prompt Engineering principles. Tailored prompts for Large Language Model (LLM) are developed for each analyzed algorithm. These prompts are explicitly designed to automatically generate human-readable explanations of algorithmic behaviors and predictions. Ultimately, the project delivers production-ready implementations and LLM integration adapters , demonstrating how the synthesis of traditional XAI methodologies with modern language models can significantly enhance the transparency, trust, and responsible deployment of AI systems.

TO DO

— TO DO

Acknowledgements

TO DO Prof. Michele Scquizzato .

TO DO

TO DO

Padova, June 2026 *Lorenzo Soligo*

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Introduction to the stage idea, goals, company and thesis organization. This chapter is meant to provide the necessary context for the reader to understand the motivation and the structure of the thesis.s

1.1 Company

Zucchetti S.p.A. is a leading Italian software company specializing in providing comprehensive IT solutions for businesses. Founded in 1978, Zucchetti has grown to become one of the largest software providers in Italy, offering a wide range of products and services that cater to various industries, including manufacturing, retail, healthcare, and public administration.

With a strong focus on innovation and customer satisfaction, Zucchetti is committed to helping organizations optimize their operations and achieve their business goals through cutting-edge technology. Exactly from this commit, the company has been actively involved in the development and implementation of ML and Artificial Intelligence (AI) solutions, aiming to enhance the capabilities of their software offerings and provide more value to their clients.

Zucchetti S.p.A. logo.

1.2 Stage idea

The internship project is centered around the exploration of XAI techniques, with a specific focus on the interpretability and comprehensibility of machine learning algorithms. The primary objective is to analyze a variety of predictive models, ranging from basic regression and classification algorithms to more complex ensemble methods, in order to evaluate their transparency and explainability. The idea is to increase the undestanding of the results of the models, and to make them more accessible to non-expert stakeholders, by leveraging advanced Prompt Engineering (PE) principles to develop tailored prompts for LLMs that can automatically generate human-readable explanations of algorithmic behaviors and predictions.

The timeline of the project is structured as follows:

Week	Task
1st	Analysis of Linear Regression, Logistic Regression, Support Vector Machines.
2nd	Analysis of Decision Trees, Random Forests, XGBoost.
3rd	Code implementation and Prompt Engineering for the analyzed algorithms.
4th	Evaluation and documentation of the results.
5th	Deep dive in Random Forests and research of real-world dataset.
6th	Analysis of Symbolic Regression.
7th	Code implementation and Prompt Engineering for the analyzed algorithms.
8th	Evaluation and documentation of the results, final report writing.

Table of the project timeline.

1.3 Project goals

The goals follow a notation to distinguish between necessary (N), desirable (D) and optional (O) goals. In the planning phase the goals were defined as follows:

Goal	Description
O-01	Complete and profound comprehension of the choosen algorithms, their mathematical foundations and their internal knowledge representation.
O-02	Comprension of interpretability and explainability techniques in machine learning.
O-03	Comprehension of algorithms design techniques.
O-04	Prompt generation for Large Language Models to generate human-readable explanations.
O-05	Comprehension and application of Prompt Engineering principles in the context of XAI.
D-01	Creation of a metric to evaluate the explainability of the analyzed algorithms.
D-02	Automatization of the analysis pipeline from dataset to LLM requests.
F-01	Application in real-world scenarios of the interpretability and explainability of Machine Learning models.
F-02	Explainability study of semi-supervised and unsupervised algorithms.

Table of project goals.

1.4 Thesis structure

The second chapter describes the basic concepts that fuel the project. The concepts spread from the mathematical foundations of the analyzed algorithms, the basics of explainability, to the principles of Prompt Engineering and LLMs.

The third chapter analyzes the ML algorithms, focusing on their capability, limitation and interpretability.

The fourth chapter describes the design and implementation of the analysis pipeline from dataset to LLM requests.

The fifth chapter discusses prompt engineering techniques and their application in the context of XAI.

The sixth chapter presents the conclusions of the study, the findings and the goals assessment.

Chapter 2

Background knowledge

In this chapter, we will provide the necessary background knowledge to understand the project. We will cover the mathematical foundations of the analyzed algorithms, the basics of explainability, and the principles of Prompt Engineering and LLMs.

2.1 Algorithmic analysis structure

The analysis of the algorithms is structured in a systematic way to ensure a comprehensive evaluation of their interpretability and comprehensibility as well as thir functioning. The structure includes the following components:

1. **Mathematical foundations**: detailed examination of the mathematical principles underlying each algorithm, including their assumptions and theoretical properties. The focus is on the understanding of the specific Objective Function that the algorithms optimize, the regularization techniques they use, and the mathematical properties that govern their behavior.
2. **Computational complexity**: analysis of the computational requirements of each algorithm, including time complexitys and scalability reguarding both training and inference.
3. **Internal representation**: exploration of how each algorithm internally represents data and makes decisions, especially regarding Categorical Features.
4. **Data assumptions**: investigation into the assumptions each algorithm makes about the data, such as linearity, independence of features, and distributional assumptions.
5. **Predictive performance and limitations**: evaluation of the predictive performance of each algorithm on relevant datasets, considering the data assumptions and mathematical foundation of the model.
6. **Metrics for prediction quality**: presentation of the most relevant metrics for evaluating the models performance, based on the task.
7. **Metrics for interpretability**: presentation of the most relevant metrics for evaluating the models interpretability, based on the task.
8. **Explainability limitations**: discussion of the limitations of explainability techniques for each algorithm, including potential pitfalls and challenges in interpreting their predictions.

2.2 Explainability

Explainability (or interpretability) of a ML model is the ability to communicate **how and why** the model made a prediction.
Contrary to intuition, explainbility it's **not a binary attribute**, but a spectrum with multiple dimensions.

2.2.1 Explainbility dimensions

1. **Intrinsic Transparency (Model-Level Interpretability)**
   The model is inherently transparent and interpretable by design. The structure directly communicates how it works, such as regression coefficients or decision tree nodes. Each prediction is fully traceable, and parameters have tangible meanings. Explanations reflect human reasoning, making them coherent from a domain perspective.
   It rappresents the best case scenario for explainability but it is often in trade-off with performance, as more complex models tend to be less transparent but more powerful. It is generally organized in tears: High, Medium and Low trasparency.
2. **Global Interpretability (Model-Level Explainability)**
   The model is comprehansible in its entirety, allowing to understand its overall behaviour and decision-making process. Such a level of understandsing it's hardly achievable and it is usually undirectly reached by comprehending the singular components of the models instead of the model as a whole.
3. **Modular Interpretability** When global interpretability is not achievable, we can still understand the model by analyzing its components separately. Exploiting the model transparency, we can understand the role of each component in the decision-making process.
   This concept is clear in the case of linear methods, where it is easy to understand the contribution of the single weights or in tree based models, where the splits and the structure of the tree can be analyzed separately.
4. **Local Interpretability (Instance-Level Explainability)** The model might be opaque and to complex to understand globally. However, for a specific prediction, we can generate an explanation that is locally interpretable. SHAP could be used in this context to explain a particular instance obtained a certain prediction.

2.2.2 Explanation properties and factors that facilitate understanding

For human explanations to be effective, they should have certain properties that facilitate understanding and usability. The following properties can be identified from the work of Robnik-Sikonja and Bohanec 2018 and Molnar [1]:

- **Contrastive explanations**

Comparative explanations that highlight the differences between instances can be more informative than absolute explanations. The instances confrontation makes the explanation more impactful, providing a direct idea of how different factors contribute to different outcomes.s

- **Cause selection**

Not all features are equally important; explanations should focus on the most influential factors rather than listing all contributing features. A small sample of the most relevant features provide a more clear explanation than a long list of all features prevventing information overload.

- **Social and contextual factors**

The social environment and the audience's expertise level should be considered when generating explanations to ensure they are appropriate and understandables.

- **Anomalies or abnormal predictions**

Highlighting anomalies or abnormal predictions can help users understand when the model is making an unusual decision, which can be crucial for trust and debugging. Features that are not largerly relevant for the majority of the predictions but that have a large impact on specific predictions should be highlighted for their capability.

- **Fidelity**

The explanation should accurately reflect the model's behavior and not introduce misleading information. A high-fidelity explanation ensures that users can trust the insights provided and make informed decisions based on them, provided that the model is accurate in its predictions.

- **General and probable**

In the absence of anomalies of specific cases, a good explanation should rely on general and probable causes that are likely to be true for most instances. This parameter can be quantified by the feature support:

$$\text{Feature Support}_i = \frac{\#\text{training instances with similar values to } \; x_i}{\#\text{total training instances}}$$

Generality can be misleading, as it may lead to explanations that are too broad and not specific enough to be useful. It's important to balance generality with the impact of the explanation, ensuring that it provides meaningful insights without being overly vague. This is why usually abnormal features are highlighted, as they can provide more specific and impactful explanations.

2.2.3 SHAP analysis

SHAP (SHapley Additive exPlanations) is a method for explaining the output of machine learning models by attributing the prediction to each feature. It is based on the concept of Shapley values from cooperative game theory, which provide a way to fairly distribute the "payout" (in this case, the prediction) among the "players" (the features) based on their contribution to the prediction. SHAP values can be used to understand the importance of each feature for a specific prediction (local interpretability) or for the model as a whole (global interpretability). SHAP provides a unified framework for interpreting various types of models and can be applied to both linear and non-linear models, making it a powerful tool for explainable AI.

SHAP library logo.

2.3 Prompt Engineering and LLMs

The following sections will present the principles used in the design of the prompts for the LLMs to generate human-readable explanations of the analyzed algorithms.

2.3.1 Expert Personas

The expert personas techiniques consists in the design of prompts that describes the role that the LLM should play in the task. This technique first should guide the model in generating a more precize, coherent and relevant output.
Though the expert personas seems to be linked to a loss of accuracy in multiple benchmarks in the recent studys by Zizhao Hu [2], the principal use of the LLMs in this project is results evaluation and text generation. The data and metrics do not have to be generated and have only to be read directly from the prompt. The paper specifically talks about *"For tasks that depend on pretrained knowledge retrieval accuracy"* [2]. It's reasonable to think that in such a context, loss of accuracy is not a critical issue. In the same paper, an alignment of behavior and style to the descripted personas is observed. This drift in behavior is critical in the task of describing the results of an algorithm in a human-readable way, as the style of the explanation is fundamental to make it accessible to non-expert stakeholders.

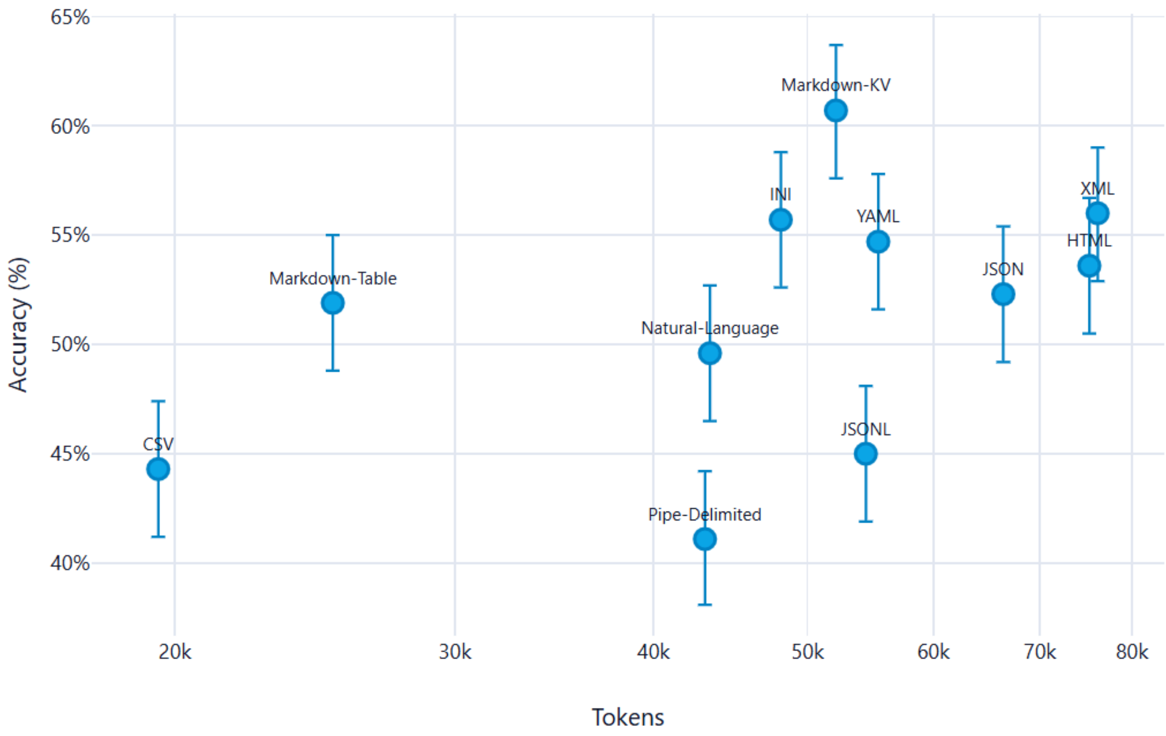
2.3.2 Familiar formats

The extensive use of LLMs have led to the emergence of some familiar formats that are widely used in the design of prompts.

The principal features that this formats present, and that makes them particularly effective, are the following:

- Widely use in training: the more a format is used in the training of the model, the more likely is to be effective in the interpretation of content in that format.
- Clear structure: the format should have a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships.
- Token efficiency: the format should be designed in a way that minimizes the number of tokens used, while still conveying all the necessary information. This is particularly important in the context of LLMs, as the number of tokens used can have a significant impact on the performance and cost of the model.

From the multitude of formats in existence, some emerged for their particular accuracy and efficiency. The most valueable one is **Markdown**, which has a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships. The use of markdown also allows to create a clear and organized structure for the generated explanations, making them more readable and accessible to non-expert stakeholders. Another accurate format is YAML, which offers a more readable formatting than JSON and XML while still being well structured.
An alternative like CSV can be used, as it is a widely used format for tabular data and is likely to be well understood by the model. However, it's important to ensure that the CSV format is properly structured and that the necessary information is included in the prompt to allow the model to generate accurate and relevant explanations.
What follows is a summarizing image whe re the 11 formats taken in exam in the study[3] are presented. The y-axis is a scale of Accuracy while the x-axis a token consumption scale. It's clear that in this particular case of study, only OpenAI's GPT-4.1 nano has been tested, Markdown emerges as the most accurate format, while mantaining good token usage.



Formats for prompting. Source: @formats-llms.

2.3.3 Chain-of-Thought

Chain of Thought (CoT) prompting is a technique that consists in the design of prompts that guide the LLMs to generate a step-by-step reasoning process to reach the final answer. This technique is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.
The additional value of th guided, step-by-step reasoning has been proven in multiple benchmarks[4], and it is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.
In the particular context of XAI and non-technical audiences because it enhances the model's ability to provide detailed and understandable explanations. The generated answer is not just a final result, but a comprehensive explanation that breaks down the reasoning process into clear and logical steps.

2.3.4 Zero-Shot prompting

This technique consists in the design of prompts that do not include any example of the expected output. It's the simplest kind of prompting because it erases the problem of having to provide training examples. Such method is particularly necessary in the context due to the fact that an example of datas and metrics for reference could be to specific and not generalized enought. An additional set of datas could also confuse the model and make it less accurate or biased on

the actual result of the analysis.
The zero-shot prompting also revealed to be quite effective with some additional techniques such as the use of Chain of Thought (CoT) prompting [5].

2.3.5 Multimodal prompt

The multimodal prompting technique consists in the design of prompts that include multiple modalities, such as text, images, tables, etc. This technique is particularly useful in the context of this project, as it allows to provide the model with a richer and more comprehensive context for generating explanations.
The use of multimodal prompts can enhance the model's ability to comprehend and analyze the results of the algorithms, as it can leverage the information provided in different formats to generate more accurate and relevant explanations. In fact, *"visual prompts effectively interact with the textual prompts, enhancing the alignment between modalities and thereby improving the model's performance on the zero-shot instruction learning"* [6].

Chapter 3

ML algorithms analysis

In this chapter, we will analyze the machine learning algorithms used in the project, looking at the mathematical foundations, the data requirements, the predictive performance, and the interpretability of the results.
This initial analysis will guide the design and implementation of the pipeline as well as the creation of the specific prompts for the LLMs to generate human-readable explanations of the results.

3.1 Linear regression

3.1.1 Mathematical model

The linear regression model is the simplest form of regression. It assumes a linear relationship between the input features and target variable. The mathematical representation of the linear regression model is:

$$y = \sum_{i=0} \beta_i x_i \forall i \in F$$

Where F is the set of features, β are the calculated weights for each of the features, and y is the target variable.
The goal of the linear regression is to find the optimal weights β that minimize the error between the predicted values and the actual target variable. This is typically done by minimizing the ordinary least squares loss function between the predicted values and the actual target variable. The optimization problem can be formulated as:

$$\hat{\beta} = \text{argmin}_{\beta_0 \dots \beta_p} \sum_{i=1}^{n} (y^{(i)} - (\beta_0 + \sum_{j=1}^{p} \beta_j x_j))^2$$

3.1.2 Time complexity

Mainly there are two approches to solve this optimization problem: the analytical solution and the iterative optimization methods (e.g. Gradient Descent (GD)). The analytical solution is given by the normal equation:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

In this formulation, X is an nxp matrix where n is the number of instances and p is the number of features. The matrix X is augmented with a column of 1s to account for the intercept term β_0. The computational complexity of the analitical solution is dominated by

the matrix multiplication and inversion operations. In fact, the cost for the moltiplication of matrixes is $O(p^2n)$ and the cost for the inversion of a pxp matrix is $O(p^3)$. Therefore, the overall computational complexity of the analytical solution is $O(p^2n+p^3)=O(p^3)$, which for vast datasets, over 10000 records, becomes prohibitive.
The iterative methods, such as GD, calculate the gradient of the loss function with respect to the weights and update the weights iteratively until convergence. The computational complexity of the gradient calculation is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. However, in practice, iterative methods can be more efficient than the analytical solution for large datasets, as they do not require matrix inversion and can converge faster with appropriate hyperparameter tuning. Other methods like Stochastic Gradient Descent (SGD) can further reduce the computational cost by approximating the gradient using a subset of the data at each iteration. On the other side, for smaller dataset, analytical solution offers convergence in a single step, without the need for hyperparameter tuning, and guarantees a global optimal solution.

For **inference**, the computational complexity is $O(p)$ per instance, as it involves a simple dot product between the feature vector and the weight vector.

3.1.3 Spacial complexity

The spacial complexity of the linear regression model is determined by the need to store the input data, the weight vector, and any intermediate matrices used in the analytical solution. Specifically, the analytical solution requires storing the nxp matrix X, the pxp matrix X^TX, and the p-dimensional weight vector β. Therefore, the overall spacial complexity of the linear regression model is dominated by the storage of the input data and the intermediate matrices, resulting in a spacial complexity of $O(np+p^2)$.
For the GD method, the spacial complexity is reduced to $O(np+p)$, as it does not require storing the intermediate matrix X^TX.
For **inference**, the computational complexity is $O(p)$ per instance, as it only stores the weight vector and the feature vector.

3.1.4 Internal representation

Linear regression represents the resulting weight of the features as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$.
Specific encoding is needed for categorical features, which are typically handled through one-hot encoding, where each category is represented as a binary feature. This can lead to an increase in the number of features and potential multicollinearity issues if not handled properly.
In regard of **explainability**, the internal representation of linear regression is straightforward and transparent, which makes it one of the most interpretable machine learning models. The weights directly indicate the strength and direction of the relationship between each feature

and the target variable. This allows for a clear understanding of how each feature contributes to the prediction, making it easier to communicate insights to stakeholders and identify important predictors in the data.

3.1.5 Data assumptions

Linear regression relies on several key assumptions about the data to ensure the validity of the model and the reliability of its predictions. These assumptions include as described by Molnar [1]:

1. **Linear constraints:** relationships between features and target must be linear. Other kind of relationships must be manually introduced and cannot be revealed automatically
2. **Residuals normality:** the residuals $\epsilon_i = y_i - \hat{y}_i$ must follow a normal distrubution. Heavy violations compromise the inference.
3. **Homoscedasticity:** residuals must have constant variance across all levels of the features. In practice, this assumption is **frequently violated**. Example: in real estate, the price of very large houses is extremely variable, while that of small houses is concentrated.
4. **Indipendence of measurements:** instances don't have to be correlated. Dipendent datas, such as time series, violate this assumption and as such should not be investigated with linear regression.
5. **Feature fisse:** le feature devono essere considerate come costanti, non soggette a errori di misurazione significativi. Se le feature contengono errore di misura, i coefficienti sono distorti (attenuation bias)
6. **Absence of multicollinearity:** Feature should not be highly correlated. Multicollinearity causes numerical instability during the inversion of $X^T X$ e weights inflation in absolute value. Using GD the geometrical properties of the loss function changes and leads to very slim vallies causing an important reduction of the learning rate.

3.1.5.1 Preprocessing

The data assumption lead to specific preprocessing steps to determine the suitability of the data for linear regression and to improve the performance of the model. These steps include:

1. **Multicollinearity identification:** calculation of the Correlation Matrix between features using the Pearson's coefficient or VIF (Variance Inflation Factor) to identify highly correlated features. $\mathrm{VIF}_j = \frac{1}{1-R_j^2}$

Where R_j^2 is the coefficient of determination for the regression of feature j against all other features. More on R_j^2 here.

3.1.6 Predictive performance and limitations

As discussed, the linear regression imposes several constraints on the data and model performance. Very good performances are achieved whenever linear relationships exist between

features and target both in accuracy and efficiency.
However, in real-world scenarios, these conditions are often violated, leading to poor predictive performance. The model is particularly sensitive to outliers, which can disproportionately influence the weights and lead to skewed predictions. Additionally, linear regression is **not suitable for capturing complex, non-linear relationships** in the data, which limits its applicability in many real-world problems where such relationships are common. Finally, the model's performance can degrade significantly when the number of features is large relative to the number of observations, leading to overfitting and poor generalization to new data. This limitation is intensified by the presence of categorical features.

3.1.7 Metrics for prediction quality

What follows is a list of most relevant metrics for evaluating the predictive performance of linear regression models, based on the task and the data assumptions.

3.1.7.1 R^2 (Determination Coefficient)

R^2 quantifies how much the model explains the total variance of the data. Il ranges from 0 to 1, where 0 is a model that cannot explain the datas and 1 is a perfect adherance.

$$R^2 = 1 - \frac{SSE}{SST}$$

Where:

- $SSE = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$ is the sum of squared residuals, quantifying the variance unexplained by the model
- $SST = \sum_{i=1}^{n}(y^{(i)} - \bar{y})^2$ is the total variance

3.1.7.2 $\bar{R}^2$ (Adjusted R²)

R^2 tends to increase with the number of features, even if those features are not useful. Adjusted R^2 penalizes the addition of non-informative features.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

Where n is the number of instances and p is the number of features.

3.1.7.3 Feature Importance (t-statistic)

$t_{\hat{\beta}_j}$ measures the statistical significance of each coefficient, calculated as the weight scaled by its standard error:

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Intuitively, the higher the absolute value of a coefficient is, the more the feature is statistically significant in predicting the target variable.

As intuitively, the hgher the variance of the coefficient is, the less the feature is significant, as the model is more uncertain about the true value of the coefficient.

3.1.7.4 p-value

p-value is a measure of the probability of "obtaining the observed data under the null hypothesis of a statistical test"[7]. In the context of linear regression, the null hypothesis is that the true coefficient β_j is equal to zero, meaning that the feature does not have a significant impact on the target variable. The p-value for each coefficient is calculated based on the t-statistic and indicates the probability of observing such an extreme value for $t_{\hat{\beta}_j}$ if the null hypothesis were true. A common convention is to consider a p-value less than 0.05 as statistically significant, suggesting that there is strong evidence against the null hypothesis and that the feature likely has a meaningful relationship with the target variable.

3.1.7.5 Mallows' Cp

Mallows'Cp is a model selection metric that balances model fit and complexity, calculated as:

$$Cp = \frac{SSE}{\hat{\sigma}^2} - n + 2p$$

Where $\hat{\sigma}^2$ is the estimate of residuals variance of the complete model. It's used to reduce the problem of overfitting, reducing the number of features.

3.1.7.6 Root Mean Squared Error (RMSE)

RMSE measures the magnitude of the errors, in the same scale as the target variable:

$$\text{RMSE} = \sqrt{\frac{SSE}{n}}$$

3.1.7.7 Mean Absolute Error (MAE)

MAE measures the average magnitude of the errors, without considering their direction:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \mid y_i - \hat{y}_i \mid$$

3.1.8 Diagnostic plots

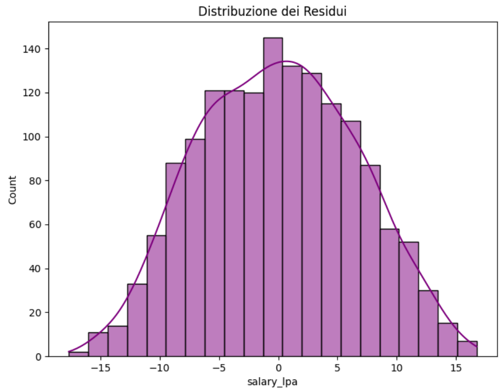
The use of diagnostic plots is crucial to visually assess the assumptions of linear regression and to identify potential issues with the model fit.

3.1.8.1 Actual vs Predicted

Scatter plot with actual values y_i on the y-axis and predicted values $\hat{y}_i$ on the x-axis. $\hat{y}_i$. If the model fits well, the points should be concentrated around the diagonal line $y = \hat{y}$. Deviations from this pattern can indicate various issues with the model fit.

3.1.8.2 Histogram of residuals distribution

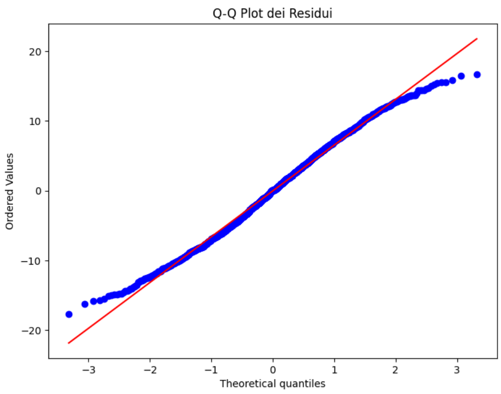
Distribution of the residuals $\epsilon_i = y_i - \hat{y}_i$. It's especially useful to identify violations of the normality assumption. A normal distribution of residuals is expected for valid inference, and deviations from this pattern can indicate issues with the model fit or the presence of outliers.



Histogram of residuals distribution example for linear regression.

3.1.8.3 Q-Q Plot (Quantile-Quantile)

Another way to check the normality of residuals is through a Q-Q plot, which compares the quantiles of the residuals to the quantiles of a normal distribution. If the residuals are normally distributed, the points in the Q-Q plot should approximately follow a straight line. Deviations from this line, especially at the tails, can indicate non-normality of the residuals, which may affect the validity of statistical inference based on the model.



Q-Q Plot of residuals example for linear regression.

3.1.8.4 Residuals vs Fitted Values

Scatter plot with fitted values $\hat{y}_i$ on the y-axis and residuals $\epsilon_i = y_i - \hat{y}_i$ on the x-axis. $\hat{y}_i$. If the model fits well, the points should be concentrated around the diagonal line $y = \hat{y}$. This plot can be used to identify violation in regression assumptions (Section 3.1.5). Increasing dispersion for example, show heteroschedasticity, while systematic patterns (e.g. a curve) indicate non-linearity that the model cannot capture.

3.1.9 Explainability and interpretability metrics

3.1.9.1 Feature Effect

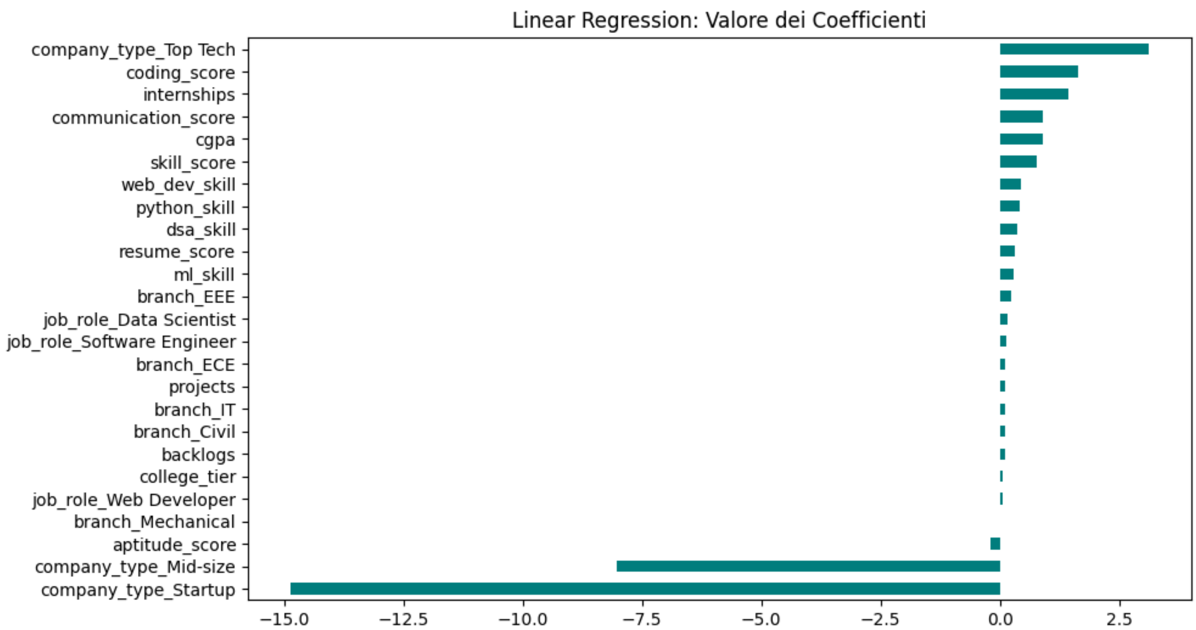
Linear regression naturally provides a direct measure of the feature effect on the target variable through the coefficients β_j. The effect of a feature x_j on the prediction for an instance i can be calculated as:

$$\text{effect}_j^{(i)} = \beta_j x_j^{(i)}$$

The standard visualization shows a box plot of the calculated effect in the quantiles 25, 50 and 75 for each feature. This allows to quickly identify the features with the most significant effect on the predictions, as well as the distribution of the effects across the dataset.
This plot results not useful if the datas are normalized, as the effect is calculated as the product of the coefficient and the feature value, and normalization can obscure the true impact of the features on the predictions.

3.1.9.2 Weight Plot

For a direct visualization of the importance of each feature, a weight plot can be used, which shows the coefficients β_j for each feature. The coefficients indicate the strength and direction of the relationship between each feature and the target variable.



Weight Plot of coefficients example for linear regression.

3.1.10 Explainability limitations

As emerged until now, linear regression is one of the most interpretable machine learning models due to its transparent internal representation and the direct relationship between features and predictions. The impact of each feature on the prediction can be easily understood through the coefficients, which indicate how much the prediction changes with a one-unit change in the feature, holding all other features constant.

What makes the model extremely interpretable make it limited in its predictive ability. Linearity of the relationships is as understandable as restrictive.

3.2 Logistic regression

3.2.1 Mathematical model

The logistic regression is a model mainly used for binary classification problems where the response variable is dichotomous. Similar to linear regression(Section 3.1) it uses a linear combination of the input features, but instead of directly outputting a continuous value, it applies a logistic function to map the output to a probability between 0 and 1.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

The output can consequently be interpreted as the probability that the input instance belongs to the positive class (y = 1). In particular, the probability that an instance belongs to the positive class is given by:

$$P(y^{(i)} = 1 \mid x^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^{p} \beta_j x_j^{(i)}))}$$

And the probability that it belongs to the negative class (y = 0) is $P(y^{(i)} = 0) = 1 - P(y^{(i)} = 1)$, following the Bernoulli distribution.

The goal of the logistic regression is to find the parameters β that best fit the data, which is done by maximizing the **likelihood** of observing the data given the parameters.

$$L(\beta; y, X) = \prod_{i=1}^{n} P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}$$

For numerical stability **log-likelihood** is usually calculated:

$$\ell(\beta) = \sum_{i=1}^{n} \left[y^{(i)} \log(P(y^{(i)} = 1)) + (1 - y^{(i)}) \log(1 - P(y^{(i)} = 1)) \right]$$

Differently from linear regression, the logistic regression does not have a closed-form solution for the parameters that maximize the likelihood so only iterative optimization algorithms like GD are available

3.2.2 Time complexity

As described in Section 3.1.2, the time complexity of GD is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. For **inference** the time complexity is again, $O(p)$ per instance.

Since the optimization is iterative, the overall time complexity can be less predictable than linear regression, especially if the data has features that lead to slow convergence (e.g., highly correlated features or features that cause the decision boundary to be close to the data points). However, in practice, logistic regression is often computationally efficient for moderate-sized datasets and can be scaled to larger datasets using SGD or mini-batch gradient descent.

3.2.3 Spacial complexity

The model stores a vector of coefficients β of size p, which requires $O(p)$ memory. During training, additional memory is needed to store the intermediate values for the optimization algorithm, which can be $O(n)$ for batch gradient descent due to the need to compute the predictions for all instances in each iteration.
For inference, the memory complexity is $O(p)$ for storing the coefficients and $O(1)$ for the input instance, leading to an overall spacial complexity of $O(p)$.

3.2.4 Internal representation

The model internally rappresents the values as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$. The presence of categorical features is solved as standard through one-hot encoding just like in linear regression.
For **explainability**, the model remains transparent since the coefficients still indicate the strength and direction of the relationship between each feature and the target variable, but the interpretation is less intuitive than in linear regression due to the non-linear transformation applied to the output by the logistic function. An increase in one feature in logistic regression leads to a multiplicative change in the odds of the positive class, rather than an additive change in the predicted value.

3.2.5 Data assumptions

Before discussing the data assumptions we need to clarify the role of the odds. The odds of an event is defined as the ratio of the probability of the event occurring to the probability of it not occurring:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)$$

This formulation more easily shows how the coefficients β_j affect the predictions.

$$\frac{\text{odds}_{x_j+1}}{\text{odds}} = \exp(\beta_j)$$

Just like in linear regression, the idea of maintaing the interpretability of the model through a linear combination of the features leads to several assumptions on the data and the model performance:

1. **Linearity of the log-odds:** $\log(\text{odds}) = \beta_0 + \sum_j \beta_j x_j$. The relationship is linear in the **log-odds space**, not in the probability space. Nonlinear relationships remain uncaptured and must be added manually.
2. **Complete separation:** if a feature perfectly separates the two classes , the model cannot identify a finite weight since the algorithm will tend to push the weight to $+\infty$ or $-\infty$, diverging. Usually the solution is to eraze this separating feature, since it is probably just a proxy for the target variable.
3. **Binomial distribution:** the response variable must follow a **Bernoulli distribution**.
4. **Independence of measurements:** observations should not be correlated. This means that the model is not suitable for temporal data without proper control, as well as for data with strong dependencies between observations.
5. **Fixed features:** features must be considered constants, without measurement error.
6. **Absence of multicollinearity:** highly correlated features cause coefficient instability. This is particularly relevant due to the fact that the iterative methods begin to oscillate and need very small learning rates (α).

Homoscedasticity is no more a requirment because the response can only be 0 or 1.

3.2.5.1 Preprocessing

To verify the suitability of logistic regression in the classification task for a given dataset, some methods can be used. The correlation matrix can be used to identify highly correlated features, which can lead to multicollinearity issues. If such features are found, they can be removed or combined to reduce redundancy and improve model stability.
For identifying anomalys in the other assumptions, plot visualitation can be used. See Section 3.2.8 for more details.

3.2.6 Predictive performance and limitations

The imposed assumptions of logistic regression can lead to good performance whenever this assumptions are met. The probabilistic interpretation gives insight on the confidence of the prediction, which is a significant advantage in many applications. Computationally wise, achieves fast training thanks to the iterative methods to solve the loss function.
However when the relationships are more complex than basic linear combinations, the predictions become less accurate. In context of unbalanced classes the training process can be dominated by the majority class, leading to poor performance on the minority class. Additionally, logistic regression as the linear regression is highly influenced by outliers, leading to very unstable predictions. There's then the limitation of the complete separation, which prevents the model to converge to a solution.

3.2.7 Metrics for prediction quality

The following is a list of metrics for evluating the predictive performances of the logistic regression in the context of the classification task.

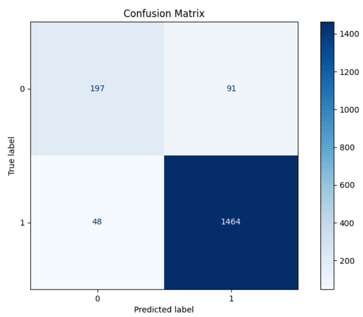
Before presenting this metrics, it is important to define some terms, used later:

- **True Positives (TP)**: instances correctly predicted as positive
- **True Negatives (TN)**: instances correctly predicted as negative
- **False Positives (FP)**: instances incorrectly predicted as positive
- **False Negatives (FN)**: instances incorrectly predicted as negative

3.2.7.1 Confusion Matrix

The confusion matrix is a table that summarizes the results of a classification task by comparing the true class labels with the predicted class labels.

The 4 different categories are TP, TN, FP and FN as described before.



Confusion matrix of a logistic regression model.

3.2.7.2 Accuracy

Accuracy measures the ratio of correct predictions to the total predictions:

$$\mathrm{ACC} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$$

It is important ot notice that in the context of unbalanced classes, accuracy can be misleading, as a model that always predicts the majority class can achieve high accuracy while performing poorly on the minority class.

3.2.7.3 Sensitivity (Recall / True Positive Rate)

The sensitivity, also known as recall or true positive rate, measures the ratio of correctly predicted positive instances to all actual positive instances:

$$\mathrm{SENS} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

3.2.7.4 Specificity (True Negative Rate)

The specificity, also known as true negative rate, measures the ratio of correctly predicted negative instances to all actual negative instances:

$$\mathrm{SPEC} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$$

Sensitivity and specificity are particularly important in contexts where the cost of false positives and false negatives is different, such as in medical diagnosis.

3.2.7.5 Precision

Precision measures the ratio of correctly predicted positive instances to all predicted positive instances:

$$\mathrm{PREC} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

Precision is crucial in scenarios where the cost of false positives is high, such as in spam detection or fraud detection.

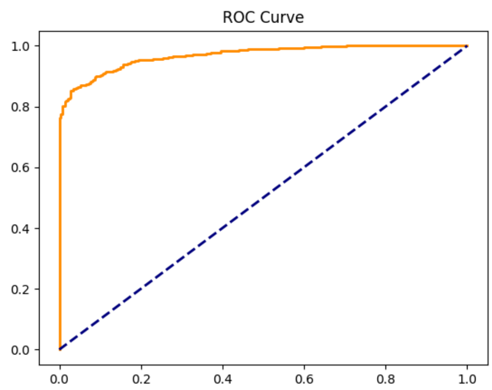
3.2.7.6 F1-Score

Out of the box, the precision and recall can be in tension, as improving one often leads to a decrease in the other. The F1-score is the harmonic mean of precision and recall, providing a single metric that balances both:

$$\mathrm{F1} = 2\frac{\mathrm{PREC} \cdot \mathrm{REC}}{\mathrm{PREC} + \mathrm{REC}}$$

It results particularly useful in unbalanced classification problems or in situations in which both false positive and false negative are costly.

3.2.7.7 ROC Curve and AUC



ROC curve of a logistic regression model.

ROC Curve is visual rappresentation of the True Positive Rate (Section 3.2.7.3) - False Positive Rate trade-off Positive Rate as the threshold of classification varies. The Sensitivity sits on the y-axis and False Positive Rate on the x-axis.
A model with good performance will have a curve that bows towards the top-left corner of the plot, indicating high sensitivity and low false positive rate across different thresholds. A model that predicts randomly will have a curve that follows the diagonal line.

AUC (Area Under the Curve) is exactly the area under the ROC curve, numerically quantifying the overall ability of the model to discriminate between the positive and negative classes. The AUC ranges from 0 to 1, with higher values indicating better performance and 0.5

representing random guessing.
Can be interpreted as the probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative instance.

3.2.7.8 Z-Statistic e p-value

The specular t-statistic for logistic regression is the Z-statistic, which measures the statistical significance of each coefficient in the model. It is calculated as:

$$Z = \frac{\beta_j}{\text{SE}\ (\beta_j)}$$

Where SE (β_j) is the standard error of the coefficient β_j. A coefficient with a large absolute value of Z indicates that the coefficient is significantly different from zero, suggesting that the corresponding feature has a significant impact on the predicted probabilities.
The p-value associated with the Z-statistic represents the probability of observing such an extreme Z value under the null hypothesis that $\beta_j = 0$. For more details on the p-value, see Section 3.1.7.4.

3.2.8 Explainability and interpretability metrics

Using plots to visualize the data and the model's predictions can help identify potential issues with the assumptions of logistic regression and provide insights into the model's performance.

3.2.8.1 Feature Effect

Similarly to Linear Regression, Section 3.1.9.1, the feature effect plot rappresent the impact of a feature on the prediction. In the logistic regression case, the effect is not on the predicted value but on the predicted probability, which is more intuitive to understand for most users.
For every feature value, calculate:

$$P(y = 1 \mid x_j = v, x_{\text{other}} = \text{media})$$

Vantaggio rispetto a LR: la trasformazione sigmoide rende visibile se l'effetto è principalmente presso certi valori della feature (grafico non è una retta, è una curva).

3.2.8.2 Weight Plot

Likewise to linear regression, the weight plot shows the coefficients of the features. However, in logistic regression, the coefficients do not directly correspond to changes in the predicted probability, but rather to changes in the log-odds of the positive class.

3.2.8.3 Odds Ratio

Direct expression of the feature impact on the odds:

$$\text{OR}_j = \exp(\beta_j)$$

It represents the multiplicative change in the odds of the positive class for a one-unit increase in the feature x_j, holding all other features constant.

3.2.9 Explainability limitations

The logistic regression, while being more interpretable than many other machine learning models, has some limitations in terms of explainability. Firstly, the interpretation of the coefficients is less intuitive than in linear regression due to the non-linear transformation applied to the output by the logistic function. An increase in one feature in logistic regression leads to a multiplicative change in the odds of the positive class, rather than an additive change in the predicted value.

This multiplicative effect is even less straightforward if we consider that the effect of a feature on the predicted probability depends on the values of all the other features, making it difficult to provide a simple explanation of the effect of a single feature without considering the context of the other features.

3.3 Support Vector Machine (SVM)

3.3.1 Mathematical model

A Support Vector Machine is a classifier that finds the optimal hyperplane that separates two classes by maximizing the distance (margin) between the hyperplane and the closest points of each class. ciascuna classe). In the case of bidimensional data, the hyperplane is a line; for three-dimensional data, it is a plane; for p-dimensional data, it is a hyperplane defined by:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p = 0$$

Where $\beta = [\beta_1, ..., \beta_p]$ is the normal vector to the hyperplane, and β_0 is the intercept.

3.3.1.1 Hard-Margin SVM (Linearly Separable Data)

In the ideal case in which the data are perfectly and **completely separable**, the goal is to find the coefficients $\beta_0, \beta_1, ..., \beta_p$ that maximize the margin. The intuition, bias, is that a hyperplane with a large margin is more **robust** to variations in the data and generalizes better to unseen data. The separation condition is then the following:

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + ... + \beta_p x_{ip}) \geq 1 \quad \forall i$$

Where $y_i \in \{-1, +1\}$ is the class label.

The distance between a point in the training set and the hyperplane is given by:

$$r_i = \frac{y_i(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij})}{|| \beta ||}$$

Where $| | \beta | | = \sqrt{\sum_{j=1}^{p} \beta_j^2}$ is the **euclidean norm** of the coefficient vector.

To maximize the margin, defined as the distance between the hyperplane and the closest points, we can express it as:

$$M = \frac{1}{|| \beta ||}$$

is equivalent to minimizing $|\,|\,\beta\,|\,|$. This formulation leads to the following optimization problem:

$$\min_{\beta,\beta_0} \frac{1}{2} \;||\; \beta \;||^2$$

Under the constraint that:

$$y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij} \right) \geq 1 \quad \forall i = 1, ..., n$$

3.3.1.2 Soft-Margin SVM (Non Linearly Separable Data)

In the realistic case where the data are **not linearly separable**, for the presence of noise, outliers, or overlapping classes, we allow some points to violate the margin. This can be achieved by intruducing **slack variables** $\xi_i \geq 0$ that measure how much a point violates the margin:

$$y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij} \right) \geq 1 - \xi_i \quad \forall i$$

Once we allow violations, we need to penalize them in the objective function to prevent the model from simply classifying all points as the majority class. This leads to the following optimization problem:

$$\min_{\beta,\beta_0,\xi} \left[\frac{1}{2} \;||\; \beta \;||^2 + C \sum_{i=1}^{n} \xi_i \right]$$

The objective function shows the importance of two components of the model. Firstly the distance of the hyperplane (first term) and secondly the violations of the margin (second term). The parameter C is a **hyperparameter of regularization** that controls the trade-off between maximizing the margin and minimizing the violations. A high value of the parameter C will prioritize minimizing the violations, rapproching the hard-margin SVM and potentially leading to a higher overfitting. A low value of C will prioritize maximizing the margin, allowing more violations.

3.3.1.3 Kernel SVM

The main limitation of linear SVM is that it only works if the data are approximately linearly separable. For data with non-linear patterns, SVM uses the **kernel trick**. The idea is to transform the data into a higher-dimensional space where the initially non linearly-separable data become linearly separable.

Computing the transformation explicitly is computationally expensive. The **kernel trick** allows us to do this implicitly.

Instead of calculating the transformation $\phi(x_i) = [f_1(x_i), f_2(x_i), ..., f_m(x_i)]$ and then calculating the product scalar $\phi(x_i)^T \phi(x_j)$, using a kernel function returns the same result directly from the original data:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

without the need to explicitly calculate ϕ.

The most common kernerls are [8]:

1. Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

This is the default kernel and corresponds to the original linear SVM. It does not perform any transformation and is suitable for linearly separable data.

2. Polinomial Kernel

$$K(x_i, x_j) = (x_i^T x_j + 1)^d$$

Transforms the data into a space of polynomials of degree d. Useful for polynomial patterns and especially precise in describing curved boundaries.

3. RBF Kernel (Radial Basis Function - Gaussian)

$$K(x_i, x_j) = \exp\left(-\gamma \mid \mid x_i - x_j \mid \mid^2\right)$$

Transforms the data into a space of infinite dimensionality. It is the **most commonly used kernel** thanks to its flexibility which makes it suitable for describing complex patterns. Also has only a single hyperparameter γ (gamma) to tune, making it easy to use.
This hyperparameter controls the influence of a single training example. The **larger** the value of γ, the **closer** other examples must be to be affected, leading to a higher risk of overfitting. Conversely, a **smaller** value of γ means that even points **far** from the decision boundary can influence it. In this case, a too small value of γ can lead to underfitting, as the model may not capture the complexity of the data.

4. Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

The sigmoid kernel resembles the activation function of a neural network and can be used in datasets where the relationship between features is expected to be *"threshold-like"* [9], splitted in to hard decision regions. However, it is less commonly used than the RBF kernel and can be more difficult to tune but sometimes can be useful instead of using a full neural network.

3.3.2 Time complexity

The time complexity for SVM training is heavily dependent on the different variants of the algorithm and the size of the dataset. In general, the training time complexity is between $O(n^2k)$ and $O(n^3k)$, where n is the number of training samples and k is the number of iterations to convergence. This is because SVM training involves solving a quadratic optimization problem, which can be computationally expensive, especially for large datasets. To make up to the cost bottleneck of kernel use in SVM, various optimizations can be employed. Approximate kernel methods, such as the Nyström method method or Random Fourier Features (RFF), can reduce the computational cost of kernel SVMs by approximating the kernel matrix by sampling a subset of the data or using random Fourier transformations. Additionally, Sequential Minimal Optimization (SMO) breaks down the optimization problem into smaller subproblems that can be solved analytically. Even with this efficient optimization algorithms, using sophisticated kernels leads to higher computational costs, balancing the better predictive performances.
On the other hand, linear SVMs with proper optimization, like Linear Programming (LP) or SGD, lower the time complexity to $O(np)$, where p is the number of features. This makes linear SVMs more scalable for large datasets, but they are limited to linear patterns.
For **inference**, the time complexity is again releted to the kernel function and the number of support vectors, which usually grows with the size of the training set and can be generally between $O(mp)$ and $O(mn)$, where m is the number of support vectors, while for linear SVMs the inference time complexity is $O(p)$, as the prediction is a simple dot product between the input features and the coefficients of the hyperplane.

3.3.3 Spacial complexity

For the memory complexity, the main bottleneck is the storage of the kernel matrix, which has a size of $O(n^2)$, making it impractical for large datasets. For linear SVMs, the memory complexity is much lower, at $O(p)$, as it only needs to store the coefficients of the hyperplane. The number of support vectors also affects the memory complexity, as each support vector requires storing its corresponding coefficient and features. In general, the memory complexity can be between $O(n^2 + mp)$ and $O(mp)$, where m is the number of support vectors.
To mitigate the memory issues that for large datasets mean impracticality, the same approximations used for time complexity can be applied, reducing the complexity of the problem in more limited and manageable subproblems.

3.3.4 Internal representation

The model rappresents the solution as a **linear combination of kernel products** of the form

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i)$$

Where:

- α_i are the dual coefficients (not directly the β_j)
- Only a fraction of the training points have $\alpha_i > 0$, these are the support vectors

For **explainability**, this representation leads to a series of disadvantages. Firstly, it is not immediately clear why a particular point is a support vector, as it depends on the complex interactions of the data and the kernel. Secondly, for non-linear kernels, the transformation ϕ is implicit and not visualizable, making it difficult to understand how the model is making decisions. Lastly, the interpretation of α_i is not intuitive, as it does not directly correspond to feature importance but rather to the influence of support vectors in the decision boundary. So, while SVM can be powerful for prediction, its internal representation poses significant challenges for explainability, especially for non-linear kernels, giving less insight into the decision-making process compared to more interpretable models like linear regression or decision trees.

3.3.5 Data assumptions

As discussed, the different variants of SVM have different characteristics, which extends to data assumptions too.

For the linear SVM, as the name suggests, the main assumption is that the data are approximately linearly separable by an hyperplane. If the assumption is not met, predictive performance decay significantly, and the model may not be able to capture the underlying patterns in the data.

Some general assumptions do exists for both linear and kernel SVMs. These include:

1. **Class balance:** SVM can be sensitive to imbalanced classes, as it focuses on maximizing the margin between classes. If one class is significantly more frequent than the other, the decision boundary may be biased towards the majority class.
2. **Feature scaling:** SVM is sensitive to the scale of the features, as it relies on the distance between data points. If features are on different scales, the model may give more weight to those with larger ranges. Therefore, it is important to standardize or normalize the features before training an SVM.

No other structural assumption emerges from the mathematical formulation giving the SVM a certain flexibility in modeling different types of data, as long as the kernel is chosen appropriately to capture the underlying patterns.

3.3.6 Predictive performance and limitations

The SVM is a powerful and versatile algorithm that is able to achieve high predictive performance, especially when the data have non-linear patterns. The Soft-margin SVM decrese the impact of outliers and a higher number of features are an improvement over the linear models, which can easily overfit in these scenarios.

However, in real-world application, with massive amount of data, the computational cost of training and inference for the kernel SVM can become prohibitive. The ideal scenario of use

is consequently for medium-sized datasets (up to tens of thousands of samples) with complex patterns that require non-linear modeling. For larger datasets, linear SVMs can be used, but they are limited to linear patterns and may not capture the complexity of the data, leading to lower predictive performance. Additionally, SVMs can be sensitive to the choice of hyperparameters which can further affect their performance. Finally, there is no direct probabilistic interpretation of the SVM outputs such as in logistic regression.
Therefore, while SVMs can be powerful for prediction, they may not always be the best choice for every problem, especially when scalability and interpretability are important considerations.

3.3.7 Metrics for prediction quality

A large number of metrics used to evaluate the predictive performance of SVMs are the same as for other classification models, such as logistic regression where they were first described in this document. These include:

- **Confusion Matrix:** Showing the number ot correct and incorrect predictions
- **Accuracy:** $(TP + TN)/(TP + TN + FP + FN)$
- **Precision:** $TP/(TP + FP)$
- **Recall/Sensitivity:** $TP/(TP + FN)$
- **Specificity:** $TN/(TN + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

The following are metrics that are more specific to SVMs.

3.3.7.1 Distance from hyperplane

This rappresent the most direct measurement of the confidence of the SVM prediction. The distance of a point x_i from the hyperplane is given by:

$$d_i = y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_j x_{ij} \right)$$

To make this distance comparable across different models and datasets, it is common to normalize it by the norm of the coefficient vector β: $d_i = \frac{d_i}{||\beta||}$
The closer the point is to the hyperplane (i.e., d_i close to 0), the less confident the prediction is, while points far from the hyperplane (i.e., d_i with large absolute value) are predicted with higher confidence.

3.3.8 Explainability and interpretability metrics

3.3.8.1 Platt Scaling

Out of the box SVM does not produce a measure of the uncertainty of its predictions, as it outputs a hard classification (+1 or −1) rather than a probability. To obtain estimates of the probability $P(y = 1 \mid x)$, a common approach is to use **Platt scaling**, which fits a logistic regression model to the SVM outputs (the distances from the hyperplane) on a validation set. The formula for Platt scaling is:

$$P(y = 1) = \frac{1}{1 + \exp(A \cdot d + B)}$$

Where A, B are parameters learned on a validation set, calibrating the distance of the hyperplane to probabilities. Although Platt scaling can provide a way to obtain probabilistic outputs from SVMs, it is important to note that the probabilities obtained through this method may not be well-calibrated, especially if the validation set used for calibration is not representative of the test set. Therefore, while Platt scaling can be useful for certain applications, it should be used with caution and its outputs should be interpreted carefully.

3.3.8.2 Support vectors and their weights (α_i)

Analysing the support vectors and their corresponding coefficients α_i can provide insights into the decision-making process of the SVM. Support vectors are the data points that lie closest to the decision boundary and have a direct influence on its position.

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x, x_i)$$

The weights α_i associated with these support vectors indicate their importance in determining the decision boundary, the higher the value of α_i, the more influence the corresponding support vector has on the decision boundary. In kernel based SVMs, their interpretation is less intuitive.

3.3.8.3 Feature importance

To assess the importance of individual features in the SVM model, we can use techniques that are model-agnostic, as SHAP values, since the SVM does not provide feature importance scores directly. The main idea is to measure how the model's predictions change when we perturb or remove a feature, which can give us insights into the importance of that feature for the model's decision-making process.

3.3.8.4 Decision boundary visualization

For low-dimensional datasets (2D or 3D), visualizing the decision boundary can provide insights into how the SVM is separating the classes. This can be done by plotting the training points, the decision boundary (the hyperplane), and the margin (the area around the hyperplane where support vectors lie). Support vectors can be highlighted to show their influence on the decision boundary.

For higher dimensional datasets, techniques like Principal Component Analysis (PCA) can be used to reduce the dimensionality and visualize the decision boundary in a lower-dimensional space, although this may not capture all the complexities of the original feature space.

3.3.9 Explainability limitations

SVM as a model has several limitations in terms of explainability, which can make it challenging to understand and interpret the decisions made by the model. These limitations are particularly pronounced when using non-linear kernels, which transform the data into a higher-dimensional space where the decision boundary is not easily visualizable. The interpretation of the weights α_i of the support vectors is not straightforward, as they do not directly correspond to feature importance but rather to the influence of the support vectors on the decision boundary.
Additionally, there is no natural way to assess feature importance directly nor to trace the reasoning process of the model, as the decision boundary is determined by a complex combination of support vectors and their corresponding weights, which can be difficult to communicate and understand, especially for non-experts.
Therefore, while SVM can be powerful for prediction, its explainability limitations should be carefully considered when choosing it for a particular application, especially when interpretability is a key requirement.

3.4 ...

Chapter 4

Design and code implementation

In this chapter we will describe the design and the implementation of the system, starting from the requirements and the technologies used, to the design and coding of the system.

4.1 Requirements

Before implementing the system, it is crucial to define what the goals and the requirements of the project are. For a more precise description this requirements are categorized in functional (F), qualitative (Q) and constraint (V) requirements, and each of them is classified as necessary (N), desirable (D) or optional (O).

Requirement	Description
RFN-1	The system must allow the user to choose the dataset to analyse
RFN-2	The system must allow the user to choose the algorithm for the analysis
RFN-3	The system must allow the user to choose the level of detail for the analysis (example SHAP/No-SHAP)
RFN-4	The system must allow the user to choose whether to execute the LLM analysis or not
RFN-5	The system must allow the user to access the result of the analysis in a clear and organized way
RFN-6	The system must allow the user to know the reason in case of an error during the execution of the analysis
RFN-7	The system must allow the user to run multiple analyses to compare the performances of different algorithms and techniques
RFO-1	The system should be accessible via graphical user interface (No-CLI)

Requirement	Description
RQN-1	The system should be open to the use of different datasets, algorithms
RQN-2	The system should be open to the use of different explainability and analysis techniques
RQN-3	The system should be open to the use of different LLMs for the analysis
RQD-1	The system should process the data and execute the analysis in a reasonable time frame, including the LLM analysis the pipeline should not take more than 15 minutes to execute

Table of requirements for the analysis pipeline.

4.2 Technologies and tools

Considering the requirements defined in the previous section and the context of the project, the following technologies and tools were chosen for the implementation of the system:

4.2.1 Python

Python has been chosen as the main programming language for the implementation of the system due to its versatility, ease of use, and the wide range of libraries and frameworks available for data analysis, machine learning, and natural language processing.

4.2.2 Markdown

Markdown has been chosen for the collection of the LLM responses to generate the final report. It was chosen for its simplicity and readability, as well as its compatibility with various tools and platforms for documentation and reporting.

4.2.3 Pandas, Matplotlib, Seaborn, Numpy, SHAP

These Python libraries have been chosen for the vast range of functionalities. Pandas for data manipulation and analysis, Matplotlib and Seaborn for data visualization, Numpy for numerical computations, and SHAP for explainability analysis (Section 2.2.3).

4.2.4 Scikit-learn

Scikit-learn provides implementations of most of the algorithms in exam, without the need to implement them from scratch, allowing to focus on the analysis and explainability aspects of the project.
Moreover, it provides a variaty of metrics for evaluating the performance of the models, which is crucial for the analysis.

The models that were not offered by scikit-learn were implemented using the original implementations provided by the authors of the algorithms, for example for XGBoost[10].

4.2.5 Optuna

Since a lot of algorithms benefits from hyperparameter tuning, Optuna was chosen for its efficiency and ease of use in automatically optimizing the hyperparameters of machine learning models.

4.2.6 Streamlit

Streamlit was chosen for the implementation of the Graphical User Interface (GUI) of the system. It allows to quickly and easily create interactive web applications for data analysis and machine learning, which is essential for providing a user-friendly interface for the system.

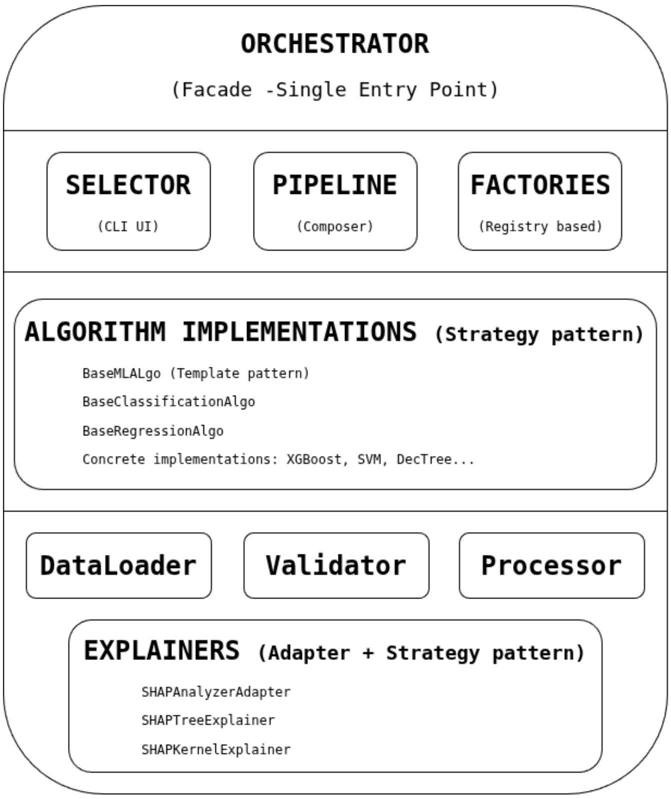
4.2.7 GitHub

GitHub was chosen for the version control of the project, for an effective codebase management.

4.2.8 Typst

Typst was chosen as principal tool for the notes and documentation. Thanks to its flexibility, modern design, powerful features, effective rendering of the PDFs, Typst

4.3 Design



The design of the entire pipeline was made with the goal of an extendable pipeline, providing the developer the possibility to easily add new algorithms, datasets, analysis techniques and LLMs.
For this reason, the design of the system is layered and modular, with each component of the pipeline being independent and interchangeable.
The main components of the system are:

Diagram of the layered architecture of the system with the most relevant components.

1. **Orchestrator**: coordinates the pipeline, invoking the different components in the correct order and passing the necessary data between them.
2. **Selector**: allows the user to choose the analysis type (single algorithm or comparative), the dataset, the algorithm, the level of detail for the analysis and whether to execute the LLM analysis or not.
3. **Algorithm**: implements the machine learning algorithm and provides the necessary functionalities for training, prediction and evaluation. All the models are created using a factory pattern, allowing to easily add new algorithms without modifying the existing code.
4. **Data Pipeline**: responsible for loading, preprocessing and splitting the data for the analysis. Every single responsibility is encapsulated in a specific class.
5. **LLM Request Manager**: manages the requests to the LLM, including the generation of prompts and the collection of responses.

4.3.1 Guiding principles

For the best possible design of the system have been used the principles of object-oriented programming and some design patterns. First of all the SOLID principles [11], [12]. In particular:

1. **Single Responsibility Principle**: each class has a single responsibility, making the code more modular and easier to maintain. A clear example is the modular structure cited just before. A concrete example is the management of the dataset. The *DataPipeline* interface describes the general structure of the data pipeline, while the *DatasetLoader* interface is responsible for loading the dataset, the *DataValidator* interface is responsible for validating the data, the *DataProcessor* interface is responsible for preprocessing the data and the *DataSplitter* interface is responsible for splitting the data into training and test sets.
   This separation allows to replace or modify each component without affecting the others, making the code more **flexible** and **maintainable**.
2. **Open/Closed Principle**: each implementation of the abstract *baseMLAlgo* can expand the possibility of the base class, adding new functionality without modifying the existing code. All the implementations inherit the basic skeleton of the base class, wrapping the specific algorithm, usually a scikit-learn estimator. The algorithm implemetation only need to implement the specific methods for fitting and metrics/plot generation following the template defined by the base class. This allows to easily add new algorithms without modifying the existing code, making the system more **extensible** and **scalable**.
3. **Liskov Substitution Principle**: the implementation of the *baseMLAlgo* substitute the abstract class, applying their version of the functions and modifying the analysis. The *orchestrator* uses the abstract class, allowing to easily switch between different implemen-

tations of the algorithms without modifying the orchestrator. The same principle has been applied in the *DataPipeline*, providing a clear interface for the data loading, validation, processing and splitting, allowing to easily **switch between different implementations** of these components without affecting the rest of the system.

4. **Interface Segregation Principle**: the interfaces of the different components are designed to be specific to their functionality, avoiding unnecessary dependencies between them. For example, the *LLMRequestManager* has a specific interface for managing the requests to the LLM, without depending on the implementation of the algorithms or the orchestrator. Same for the *DataPipeline* and the *Explainer* components, which are designed to be indipendent and modular, allowing to easily replace or modify each component without affecting the others. This design promotes **loose coupling** and **high cohesion** in the codebase, making it easier to maintain and extend.

5. **Dependency Inversion Principle**: the high-level modules (*orchestrator*) do not depend on low-level modules (algorithms, LLM request manager), but both depend on abstractions. For example, the orchestrator depends on the abstract *baseMLAlgo* and *LLMRequestManager*, allowing to easily switch between different implementations of the algorithms and the LLM request manager without modifying the orchestrator.

4.3.2 Applied design patterns

The design of the system also incorporates some design patterns to solve common problems and improve the structure of the code. In particular, the following design patterns were applied:

4.3.2.1 Strategy

The strategy pattern was applied to the implementation of the algorithms, allowing to easily switch between different algorithms without modifying the code of the *orchestrator*. Each algorithm implements the same interface defined by the abstract *baseMLAlgo*, selecting the appropriate strategy at runtime.

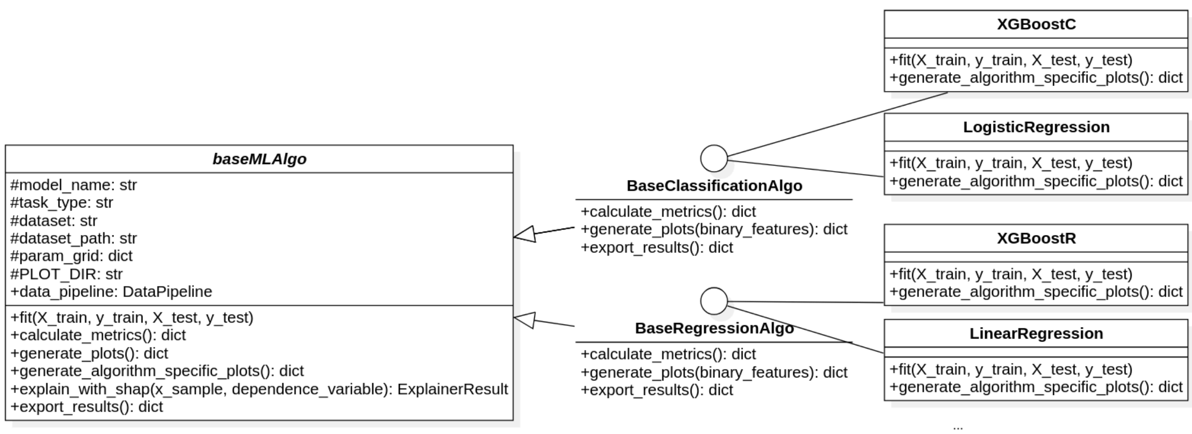


Diagram of the strategy pattern applied to the implementation of the algorithms with XGBoost, Logistic and linear regression as examples of the concrete algorithms.

4.3.2.2 Template method

The skeleton of the pipeline is defined in *BasePipeline* describing the overall structure of the analysis process. The concrete *DefaultPipeline* implements the template method, providing specific implementations for each step. The same pattern has been used in the *DataPipeline* and *Explainer* components as they share the same necessity for defined structure with possibility for customization of the specific steps.

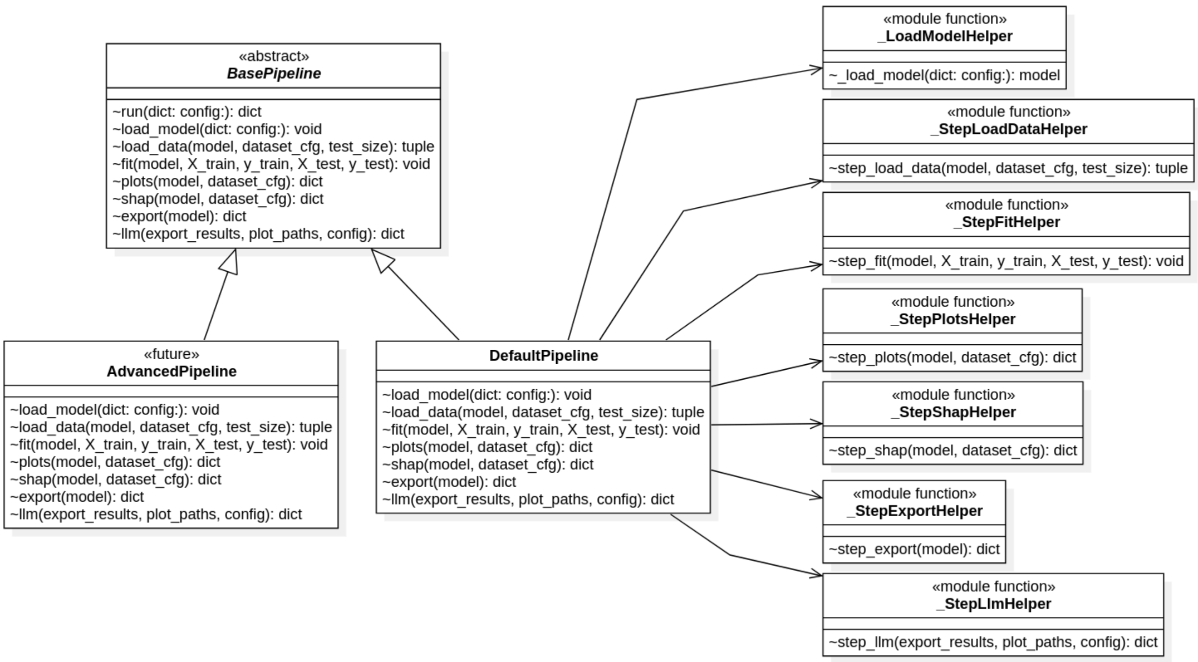


Diagram of the template method pattern applied to the implementation of the pipeline.

4.3.2.3 Factory

The factory pattern was applied in the algorithms instantiation via a *ModelFactory*, allowing to easily create instances of the algorithms without exposing the instantiation logic to the client code. This allows to easily add new algorithms without modifying the existing code and dynamically loading them, making the system more **extensible** and **scalable**.

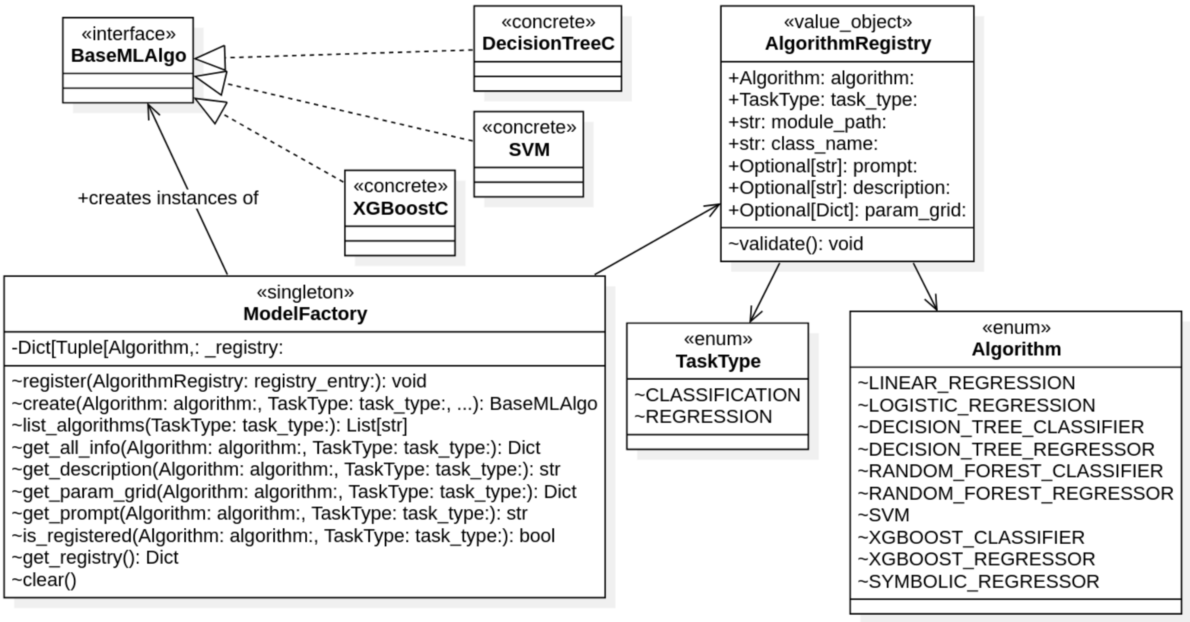


Diagram of the factory pattern applied to the implementation of the algorithms and ModelFactory.

4.3.2.4 Registry

To centrally manage the registered algorithms, a registry pattern was applied, improved by a masisve use of value objects to manage the informations about the algorithms. The validity of the registered algorithms is consequently guaranteed and it simplifies the process of adding new algorithms to the system, as it only requires to create a new class that implements the *baseMLAlgo* interface and register it in the *ModelFactory* without modifying the existing code. This design promotes **extensibility** and **maintainability** of the codebase.

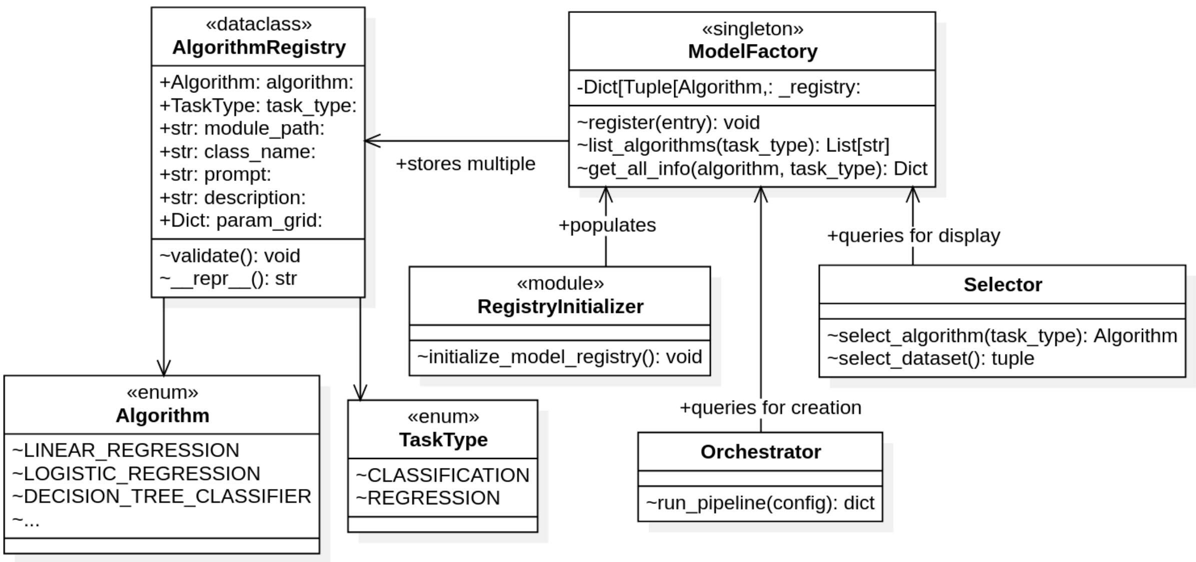


Diagram of the registry pattern applied to the implementation of the algorithms and its use in the context of the system.

4.3.2.5 Adapter

To compute the SHAP values for the algorithms, some adapting is needed to fit the specific requirements of the SHAP library. For this reason, an adapter pattern was applied to the *Explainer*, allowing to easily adapt the algorithms to the requirements of the SHAP library without modifying the existing code while maintaining a consistent interface. This design promotes **flexibility** and **reusability** of the codebase.

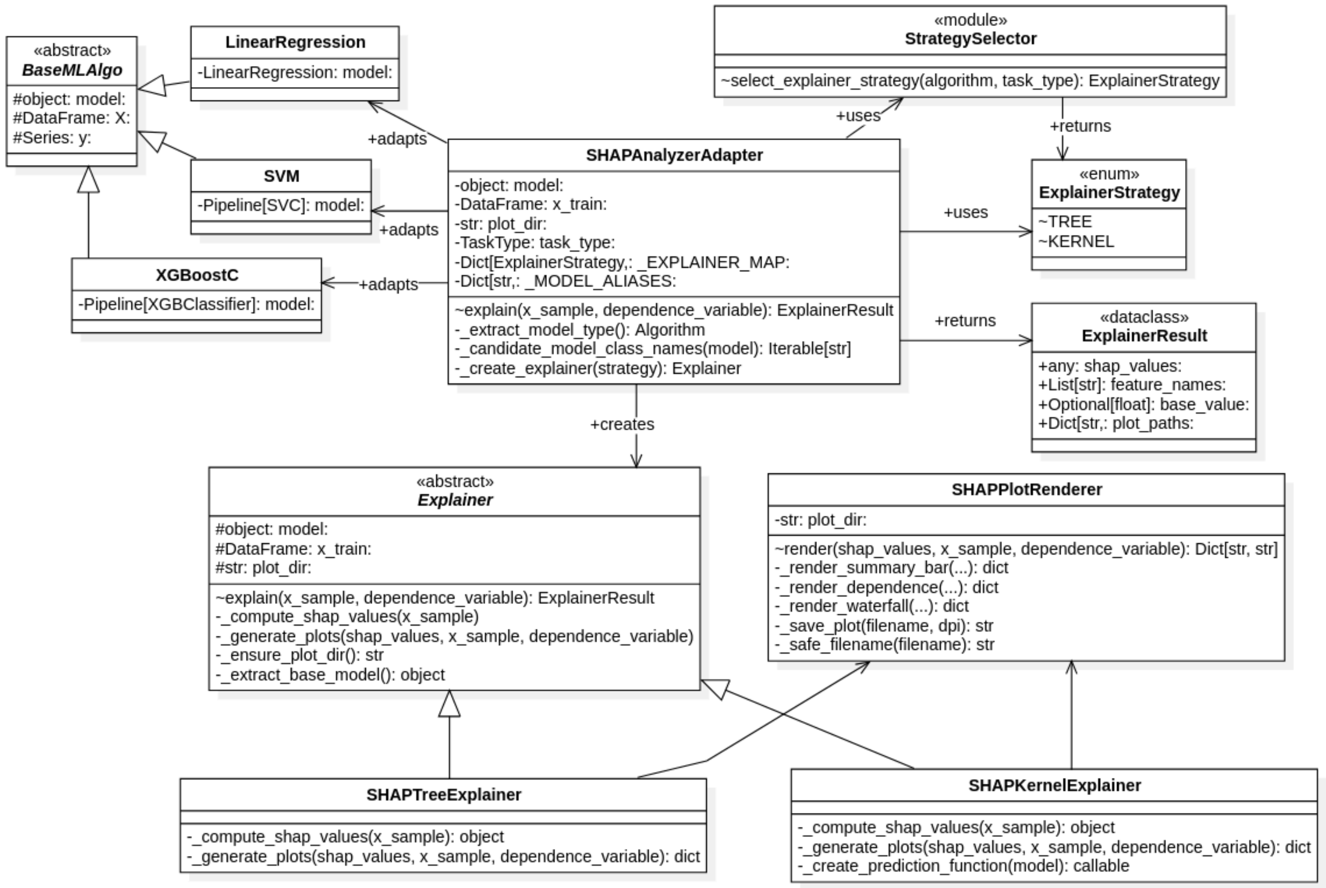


Diagram of the adapter pattern applied to the implementation of the algorithms and its use in the context of the system.

4.3.2.6 Facade

The facade pattern was applied to the *orchestrator*, providing a single entrance *run_pipeline()* for algorithm initialization, result aggregation and management of the entire process of generating prompts, sending requests to the LLM and collecting responses, hiding the complexity of the underlying implementation from the rest of the system.

4.4 Code implementation

The resulting product of the design process is consequently structured in a modular and layered way, with clear interfaces and separation of concerns between the different components.

The implementation of the system follows the design principles and patterns described in the previous sections, resulting in a codebase that is **extensible**, **maintainable** and **scalable**.

4.4.1 Extension of the system

The process of extending the system with new algorithms, datasets and analysis techniques follows a clear and structured process, which allows to easily add new components to the system without modifying the existing code. The process is as follows:

1. **New algorithm**

STEP 1

NewAlgo.py

Describe the new model following "baseMLAlgo" interface and its template method using the abstact classes as guidelines.

STEP 2

registryInitializer.py

Register the new algorithm in the "RegistryInitializer" to make it available in the system.

STEP 3

Explainer.py

Add an entry in the "Explainer" strategy using the updated registry for type safety.

2. **New dataset manipulation**

dataset_config.py

If what is being added is a new dataset, create a new entry in the "dataset_config", describing the dataset, the task, the objective and other relevant information for the analysis.

dataLoader.py

If what is being added is a new dataset source, create a new class that implements the "DataLoader" interface, providing the necessary methods for loading the new dataset.

dataValidator.py

If what is being added is a validation rule, create a new class that implements the "DataValidator" interface, providing the necessary methods for validating the new dataset.

dataProcessor.py

If what is being added is a new preprocessing step, create a new class that implements the "DataProcessor" interface, providing the necessary methods for preprocessing the new dataset.

3. **New SHAP analysis technique**

STEP 1

NewSHAP.py

Create a new class that implements the "SHAPExplainer" interface, providing the necessary methods for computing the SHAP values using the new technique.

STEP 2

strategy.py

Define in the "strategy" which algorithm can use the new SHAP technique, using the algorithms registry for type safety.

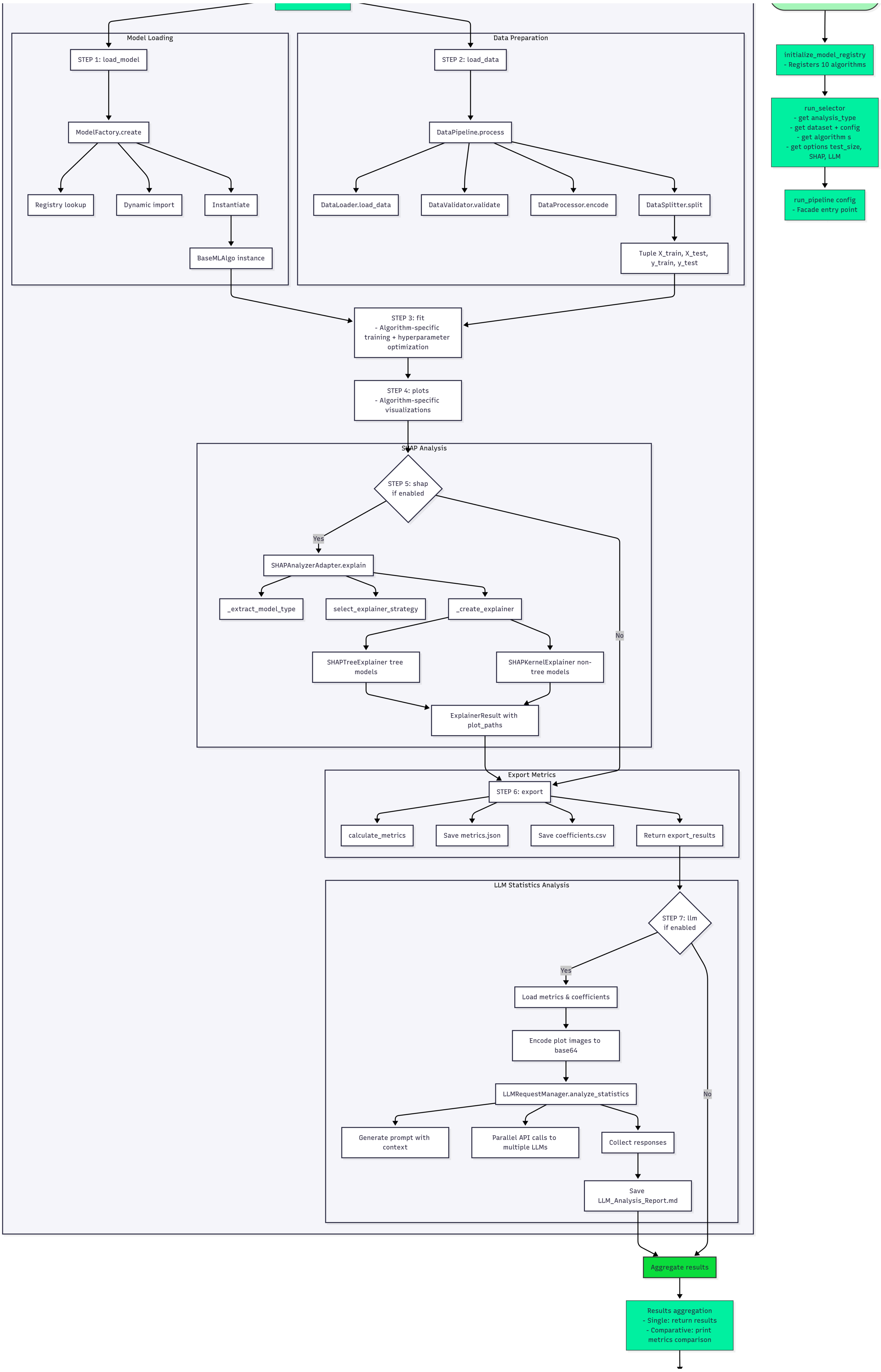
STEP 3

plot_renderer.py

Add eventually new methods for rendering the SHAP values.

4.4.2 Flow of the system

For a understanding of how the system works, the flow of the pipeline is described in the following diagram, showing the main steps of the process and how the different components interact with each other.



Model Loading
STEP 1: load_model
ModelFactory.create
Registry lookup
Dynamic import
Instantiate
BaseMLAlgo instance
Data Preparation
STEP 2: load_data
DataPipeline.process
DataLoader.load_data
DataValidator.validate
DataProcessor.encode
DataSplitter.split
Tuple X_train, X_test, y_train, y_test
STEP 3: fit - Algorithm-specific training + hyperparameter optimization
STEP 4: plots - Algorithm-specific visualizations
SHAP Analysis
STEP 5: shap if enabled
Yes
No
SHAPAnalyzerAdapter.explain
_extract_model_type
select_explainer_strategy
_create_explainer
SHAPTreeExplainer tree models
SHAPKernelExplainer non-tree models
ExplainerResult with plot_paths
Export Metrics
STEP 6: export
calculate_metrics
Save metrics.json
Save coefficients.csv
Return export_results
LLM Statistics Analysis
STEP 7: llm if enabled
Yes
No
Load metrics & coefficients
Encode plot images to base64
LLMRequestManager.analyze_statistics
Generate prompt with context
Parallel API calls to multiple LLMs
Collect responses
Save LLM_Analysis_Report.md
Aggregate results
Results aggregation - Single: return results - Comparative: print metrics comparison
initialize_model_registry - Registers 10 algorithms
run_selector - get analysis_type - get dataset + config - get algorithm s - get options test_size, SHAP, LLM
run_pipeline config - Facade entry point

Diagram of the flow of the system, showing the main steps of the process and how the different components interact with each other.

Chapter 5

Prompt Engineering

Breve introduzione al capitolo

Chapter 6

Conclusion

6.1 Consuntivo finale

6.2 Raggiungimento degli obiettivi

6.3 Conoscenze acquisite

6.4 Valutazione personale

Bibliography

[1] Christoph Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Leanpub, 2025.

[2] Zizhao Hu, Mohammad Rostami, and Jesse Thomason, "Expert Personas Improve LLM Alignment but Damage Accuracy: Bootstrapping Intent-Based Persona Routing with PRISM," 2026.

[3] "Which Table Format Do LLMs Understand Best?." [Online]. Available: https://www.improvingagents.com/blog/best-input-data-format-for-llms/

[4] Jasonn Wei *et al.*, "Chain of Thought Prompting Elicits Reasoning in Large Language Models," 2023.

[5] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, "Large Language Models are Zero-Shot Reasoners," 2023.

[6] Taowen Wang *et al.*, "M^2PT: Multimodal Prompt Tuning for Zero-Shot Instruction Learning," 2024.

[7] "P-Value: What It Is, How to Calculate It, and Examples." [Online]. Available: https://www.investopedia.com/terms/p/p-value.asp

[8] "Major Kernel Functions in Support Vector Machine (SVM)." [Online]. Available: https://www.geeksforgeeks.org/machine-learning/major-kernel-functions-in-support-vector-machine-svm/

[9] Michal Mikulasi, "What Is the Sigmoid Kernel? (And Why It Feels a Bit Like Deep Learning)." [Online]. Available: https://medium.com/@michalmikuli/what-is-the-sigmoid-kernel-and-why-it-feels-a-bit-like-deep-learning-aa0210055c4e

[10] Tianqi Chen and Carlos Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD '16*, 2016.

[11] Robert C. Martin, "Design principles and design patterns," 2002. [Online]. Available: https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf

[12] Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson, 2008.

[13] Zhen "Leo" Liu, *Artificial Intelligence for Engineers: Basics and Implementations*. Springer, 2024.

[14] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz, "A Comparative Analysis of XGBoost," *Artificial Intelligence Review*, vol. 54, no. 3, 2019.

[15] Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[16] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*. Springer, 2023.

[17] V. K. Xidong Wu, *Top 10 Algorithms in Data Mining*. CRC Press, 2009.

[18] Lior Rokach and Oded Maimon, *Data Mining with Decision Trees: Theory and Applications*. World Scientific Publishing, 2008.

[19] "SHAP Documentation." [Online]. Available: https://shap.readthedocs.io/en/latest/

[20] Xavier Amatriain, "Prompt Design and Engineering: Introduction and Advanced Methods," 2024.

Chapter 7

Glossary

***AI* – Artificial Intelligence**: Field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, problem-solving, and understanding natural language. 1

***categorical features* – Categorical Features**: Features in a dataset that represent discrete categories or groups, rather than continuous numerical values, and often require special handling in machine learning algorithms. 5, 14, 16, 21

***classification* – Classification**: A type of machine learning problem in which the goal is to predict discrete class labels based on input features. i, 2

***computational complexity* – Computational complexity**: A measure of the amount of computational resources (time and space) that an algorithm requires to run, often expressed in terms of the size of the input data. 5, 13, 14, 14, 14, 14

***correlation matrix* – Correlation Matrix**: A table showing the correlation coefficients between pairs of features in a dataset, used to identify relationships and potential multi-collinearity. 15, 22

***XAI* – Explainable Artificial Intelligence**: A field of artificial intelligence that focuses on creating models and systems that can provide clear and understandable explanations for their decisions and actions. i, i, 2, 10

***FN* – False Negatives**: Instances incorrectly predicted as negative. 23, 23

***FP* – False Positives**: Instances incorrectly predicted as positive. 23, 23

***GD* – Gradient Descent**: An optimization algorithm used to minimize the objective function in machine learning models by iteratively adjusting the model's parameters in the direction of the steepest descent. 13, 14, 14, 15, 20, 20

***GUI* – Graphical User Interface**: A user interface that allows users to interact with a computer system through graphical elements such as windows, icons, and buttons, rather than text-based commands. 37

***LLM* – Large Language Model**: A type of artificial intelligence model that is trained on vast amounts of text data to understand and generate human-like language. i, i, 2, 8, 8, 8, 10, 13, 37, 38, 38, 39, 42

***LP* – Linear Programming**: A mathematical optimization technique used to find the best outcome in a model whose requirements are represented by linear relationships, often used in machine learning for training linear SVMs and other models. 29

***MAE* – Mean Absolute Error**: A common metric for evaluating the performance of regression models, calculated as the average of the absolute differences between predicted and actual values. iv, 17

***margin* – Margin**: The distance between the decision boundary (hyperplane) and the closest data points in a Support Vector Machine, which the algorithm seeks to maximize for better generalization. 26, 26, 27

***ML* – Machine Learning**: A subset of artificial intelligence that involves training algorithms to learn patterns from data and make predictions or decisions without being explicitly programmed. i, 1

Nyström method: A technique used to approximate large kernel matrices in machine learning, which allows for efficient training of models like Support Vector Machines on large datasets by sampling a subset of the data and using it to construct a low-rank approximation of the kernel matrix. 29

***objective_function* – Objective Function**: A mathematical function that an algorithm optimizes during training, which defines the goal of the learning process and guides the model's adjustments to minimize error or maximize performance. 5

outlier: A data point that deviates significantly from the majority of the data, which can have a disproportionate influence on machine learning models and may require special handling. 16, 30

***PCA* – Principal Component Analysis**: A dimensionality reduction technique that transforms a dataset into a new coordinate system, where the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on, allowing for efficient visualization and analysis of high-dimensional data. 32

***PE* – Prompt Engineering**: The process of designing and optimizing prompts to effectively communicate with language models and elicit desired responses. 2

***RFF* – Random Fourier Features**: A technique used to approximate kernel functions in machine learning, allowing for efficient training of models like Support Vector Machines on large datasets by mapping input data into a higher-dimensional space using random Fourier transformations. 29

***regression* – Regression**: A type of machine learning problem in which the goal is to predict continuous numerical values based on input features. i, 2

***RMSE* – Root Mean Squared Error**: A common metric for evaluating the performance of regression models, calculated as the square root of the average of the squared differences between predicted and actual values. iv, 17

***SMO* – Sequential Minimal Optimization**: An algorithm used to solve the optimization problem in Support Vector Machines by breaking it down into smaller subproblems that can be solved analytically, which allows for efficient training of SVMs on large datasets without the need for complex numerical optimization techniques. 29

SHAP **– SHapley Additive exPlanations**: A method for explaining the output of machine learning models by attributing the prediction to each feature. i

SGD **– Stochastic Gradient Descent**: An optimization algorithm that updates the model's parameters using a randomly selected subset of the training data, which can lead to faster convergence and better generalization. 14, 21, 29

TN **– True Negatives**: Instances correctly predicted as negative. 23, 23

TP **– True Positives**: Instances correctly predicted as positive. 23, 23